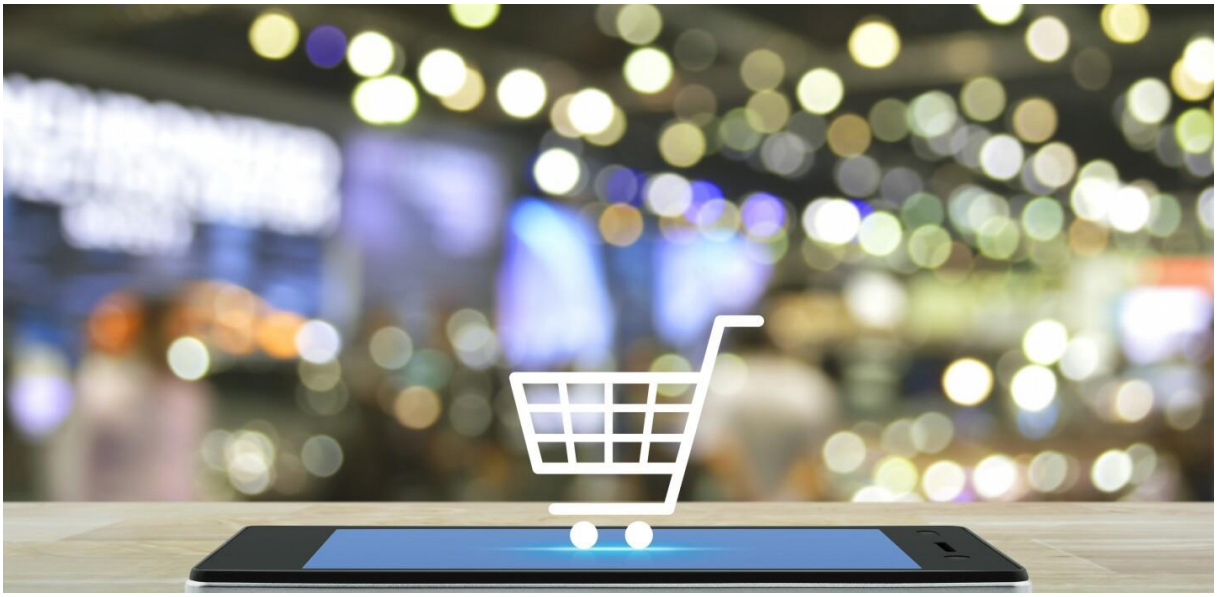
IUT de Paris - Rives de Seine
Université Paris Cité

BUT 2 SF – FA VCOD

05/03/2026

COLLECTE AUTOMATISEE DE DONNEES WEB

DIGITALISATION DES COMMERCES PARISIENS



Christella DAVID

Santatriniaina RASOLOFOMANANA

SOMMAIRE

I- INTRODUCTION

II- METHODOLOGIE

1) Récupération des coordonnées GPS (API Adresse)
2) Recherche du site web (Google Maps)
3) Possibilité de livraison

III- RESULTATS

1) Coordonnées GPS
2) Site web des commerces
3) Possibilités de livraison

IV- CONCLUSION

I- INTRODUCTION

Dans le cadre de cette SAE, nous nous sommes intéressées à la **digitalisation des commerces parisiens**. En effet, beaucoup de commerces ont un site web ou proposent des services en ligne, comme la livraison. L'APUR, l'Atelier parisien d'urbanisme, suit ces évolutions dans la base **BDCOM2023**, qui rassemble les informations sur les commerces situés à Paris.

À partir de ces données, le but du projet était d'**enrichir la base** avec des éléments qui ne sont pas directement fournis : les coordonnées GPS, l'existence d'un site internet pour chaque commerce, et la possibilité de se faire livrer.
Pour cela, nous avons utilisé les outils vus en cours, notamment l'API Adresse pour récupérer la géolocalisation, et une recherche automatisée sur Google Maps pour repérer les sites web.

II- METHODOLOGIE

Dans cette partie, on retrouve les différentes étapes que nous avons suivies pour enrichir les données des commerces parisiens. Nous avons commencé par la récupération des coordonnées GPS, puis par la recherche du site web des commerces.

1) Récupération des coordonnées GPS (API Adresse)

Nous avons d'abord cherché à récupérer la latitude et la longitude de chaque commerce parisien.

Étapes suivies :

- **Importation des bibliothèques**
  Nous avons d'abord chargé toutes les bibliothèques nécessaires à notre projet, notamment pandas, selenium, bs4, time et requests.

```
import pandas as pd
from selenium import webdriver
from bs4 import BeautifulSoup
import time
import requests
```

- **Lecture du fichier**
  Nous avons chargé un fichier de **1000 commerces**, comme recommandé, pour éviter de surcharger l'API. Nous avons alors renseigné le chemin du fichier pour accéder à la base de données.

```
path = r"C:\Users\ChristellaDAVID\OneDrive - STARTUM CFA\Bureau\Stella\BDCOM_2023.xls"
bdcom = pd.read_csv(path, nrows=1000)
```

- **Extraction des colonnes utiles**
  Pour construire une adresse complète, nous avons sélectionné les colonnes nécessaires :
  - Identifiant : `OBJECTID`
  - numéro de voie : `num`
  - type de voie : `typ_voie`

- nom de la voie : `lib_voie`
- arrondissement : `arro`

```
exporter_adresse = bdcom[["OBJECTID", "num", "let", "typ_voie", "lib_voie", "arro"]]
exporter_adresse.to_csv("exporter_geo.csv", index=False)
```

- **Envoi des adresses à l'API**

  Chaque adresse a été envoyée automatiquement à l'API.
  L'API renvoie un fichier CSV contenant, pour chaque ligne :

  - `la latitude`
  - `la longitude`

```
url = "https://api-adresse.data.gouv.fr/search/csv/"
fichier = open("exporter_geo.csv", "rb")
r = requests.post(url, files={"data": fichier})


fichier_resultat = open("exporter_geo_sorties.csv", "w", encoding="utf-8")
fichier_resultat.write(r.text)
fichier_resultat.close()
```

- **Jointure avec les données d'origine**

  Une fois le fichier reçu, nous l'avons **fusionné** avec le tableau initial grâce à la colonne qui identifie chaque ligne. Nous avons effectué une jointure à gauche. Grâce à ce code, la latitude et la longitude ont été ajoutées.

Sur cette partie, nous avons directement rajouté les colonnes website et livraison, pour éviter de perdre du temps, et nous les avons mis dans un même fichier csv nommé : `BDCOM2023_complet_gps.csv`

```
geo = pd.read_csv("exporter_geo_sorties.csv")

# Fusion
bcom_geo = bdcom.merge(geo, on="OBJECTID", how="left")

# Ajout des colonnes pour le site web et la livraison
bcom_geo["website"] = None
bcom_geo["livraison_possible"] = "Inconnu"

bcom_geo.to_csv("BDCOM2023_complet_gps.csv", index=False)
```

2) Recherche du site web (Google Maps)

Après avoir ajouté les coordonnées GPS, nous avons cherché à savoir si chaque commerce possédait un site web officiel. Etant donné que la colonne website a déjà été créée, nous commençons directement par la construction de l'URL google maps.

Étapes suivies :

- **Formatage des éléments d'adresse**

  Nous avons développé une fonction pour formater correctement les éléments d'adresse, en évitant les erreurs liées aux valeurs manquantes ou aux numéros enregistrés en décimal. L'objectif était d'obtenir une adresse propre pour la

suite du traitement. Nous avons mis quelques commentaires pour pouvoir suivre la logique de la fonction.

```
def f(adresse_element, est_dernier_element=False):
    # Initialisation
    resultat_formate = ""
    # Vérification que l'élément n'est pas une valeur manquante_
    if not(pd.isna(adresse_element)):
        # Si l'élément est un nombre décimal, le convertir en entier puis en chaîne
        if isinstance(adresse_element, float):
            resultat_formate += str(int(adresse_element))
        # Sinon, simplement convertir en chaîne
        else:
            resultat_formate += str(adresse_element)
        if not(est_dernier_element):
            resultat_formate += "+"
    return resultat_formate
```

- **Configuration du traitement par lots**

  Nous avons configuré un traitement par lots parce que nous avions près de 1000 commerces à analyser, et envoyer toutes les requêtes d'un seul coup aurait provoqué des blocages ou un ralentissement du navigateur. Pour éviter cela, nous avons découpé le travail en paquets de 50 commerces, avec des pauses entre les lots et entre chaque requête afin de respecter les limites de Google Maps. Ces pauses reposent sur la fonction *time.sleep()*, et c'est pour cette raison que nous avons chargé la bibliothèque *time* au début du script.

```
# traitement par lots
total_commerces = len(bdcom)
taille_lot = 50
pause_entre_lots = 5
pause_entre_requetes = 0.2

# navigateur
driver = webdriver.Chrome()
base_url = "https://www.google.com/maps/search/"

# stockage des sites web
sites = [None] * total_commerces
```

Pour les prochaines étapes, les codes sont mis dans une boucle pour le traitement par lots.

```
for debut_lot in range(0, total_commerces, taille_lot):
    fin_lot = min(debut_lot + taille_lot, total_commerces)
```

- **Construction de l'URL Google Maps**

  À chaque commerce, nous reconstruisons l'adresse pour faire une recherche Google Maps, ce code se trouve toujours dans la boucle du traitement par lots. Grâce à notre fonction f(), chaque élément d'adresse est formaté proprement et séparé par "+". Cela donne une URL qui permet de lancer une recherche précise sur Google Maps.

```
for i in range(debut_lot, fin_lot):
    try:
        commerce = bdcom.iloc[i]

        place_info = ""
        if not pd.isna(commerce["ens"]):
            place_info += f(commerce["ens"])

        place_info += f(commerce["num"]) + f(commerce["let"]) + \
                      f(commerce["typ_voie"]) + f(commerce["lib_voie"]) + \
                      f(commerce["arro"], True)

        # URL complète
        url = base_url + place_info
```

- **Ouverture de la page avec Selenium**
  Toujours dans la boucle, l'objectif était d'automatiser l'ouverture des résultats pour ensuite analyser la page. Nous laissons volontairement un petit délai après l'ouverture pour que la page soit bien chargée.

```
driver.get(url)
time.sleep(pause_entre_requetes)
```

- **Extraction du HTML**
  Une fois la page chargée, nous avons récupéré le code HTML avec BeautifulSoup pour pouvoir analyser les éléments affichés par Google Maps. Nous avons utilisé ce sélecteur pour rechercher le bouton « site web » : `soup.select("a[aria-label*='Site Web']")`

```
html = driver.page_source
soup = BeautifulSoup(html, 'html.parser')
resultat = soup.select("a[aria-label*='Site Web']")
```

- **Stockage du résultat**
  Pour finir, nous avons stocké le résultat dans la colonne website créée au début.

```
if resultat:
    website = resultat[0]["href"]
    sites[i] = website
```

3) Possibilité de livraison

Nous avons ici mis en place une méthode simple pour repérer automatiquement si un commerce propose la livraison. L'idée est d'utiliser la même page Google Maps que pour la recherche du site web, puis d'analyser les liens présents dans le HTML.

Google Maps affiche parfois des liens directs vers des plateformes comme UberEats, Deliveroo ou Just Eat. Nous avons donc parcouru tous les liens de la page et nous avons vérifié si certains contenaient des mots-clés caractéristiques, par exemple "livraison", "delivery", "uber eats", "deliveroo", "just eat".
Si au moins un de ces liens est présent, cela signifie que le commerce propose la livraison, et la plateforme correspondante est enregistrée. Sinon, la valeur “Non” est attribuée.

```
        try:
            driver.get(website)
            time.sleep(pause_entre_requetes)
            page_content = driver.page_source.lower()
            # Recherche de mots liés à la livraison
            if any(keyword in page_content for keyword in ["livraison", "delivery",
                                                           "uber eats", "deliveroo", "just eat"]):
                bcom_geo.loc[i, "livraison_possible"] = "Oui"
            else:
                bcom_geo.loc[i, "livraison_possible"] = "Non"
        except:
            pass
except Exception as e:
    pass
```

MISE A JOUR DES RESULTATS

```
# Fermeture du navigateur
driver.close()

# base de donnée finale
bcom_geo.to_csv("BDCOM2023_final.csv", index=False)
```

III- RESULTATS

Après avoir appliqué toutes les étapes, nous avons pu enrichir les données des commerces avec les informations demandées : les coordonnées GPS, les sites web, et la livraison.

1) Coordonnées GPS

Sur la base que nous avons traitée, la grande majorité des commerces a pu être géolocalisée correctement. Les colonnes latitude et longitude sont donc bien remplies dans presque toutes les lignes, et cela qui nous permet d’avoir une localisation précise des établissements. Seules quelques adresses trop imprécises n’ont pas pu être converties, mais cela reste très limité.

2) Site web des commerces

Concernant les sites web, nous avons réussi à en identifier un certain nombre. Dès que la fiche du commerce affichait un lien, nous avons pu l’ajouter directement dans la base. Tous les commerces n’ont pas de site officiel, ce qui explique certaines cases vides. On observe quand même une présence en ligne assez variée : les commerces plus “modernes” ou appartenant à des chaînes fournissent plus souvent un lien vers un site internet.

3) Possibilités de livraison

Pour la livraison, nous avons pu repérer plusieurs cas où le commerce était associé à des plateformes comme UberEats, Deliveroo ou Just Eat. Ces situations ne sont pas majoritaires, mais elles existent, surtout pour les commerces liés à la restauration. Quand aucun lien de ce type n'apparaissait, nous avons simplement noté que la livraison n'était pas proposée. Cela permet déjà d'avoir une première idée de l'usage des services de livraison chez les commerçants parisiens.

IV- CONCLUSION

À la fin de ce travail, nous arrivons à une base de données qui est vraiment plus complète qu'au départ. Nous avons pu ajouter les coordonnées GPS pour presque tous les commerces, ce qui permet de bien les situer. Nous avons aussi repéré des sites web quand ils étaient visibles, ce qui donne une première idée de ceux qui ont une présence en ligne. Pour la livraison, nous avons réussi à repérer quelques liens vers des plateformes, surtout pour les commerces liés à la restauration.

Nos difficultés :

Tout n'a pas été simple pour autant. Certaines adresses n'étaient pas assez précises pour être géolocalisées correctement. L'accès à Google Maps a aussi posé quelques difficultés, en effet nous avons eu un problème sur le chargement de Selinium à cause des règles réseaux et bannières cookies, ce qui a rendu la collecte plus lente que prévu. Et bien sûr, beaucoup de commerces n'affichent pas toujours leurs informations en ligne, ce qui limite un peu ce qu'on peut récupérer. Le temps de chargement des données aussi a posé quelques soucis comme nous avons dû inclure des pauses (time.sleep()) pour laisser le temps aux pages de se charger et éviter d'être bloqués par Google, ce qui a fait qu'il nous a fallu une trentaine de minutes pour exécuter tout le code et charger la table finale.

Aujourd'hui, les habitudes changent vite et le numérique prend de plus en plus de place dans les commerces. Les données que nous avons ajoutées permettent déjà de mieux comprendre cette évolution. Pour aller plus loin, on pourrait analyser plus de commerces ou comparer plusieurs années pour voir comment la situation change.